

Trabalho Prático 2 - Sistemas Operacionais

1 Introdução

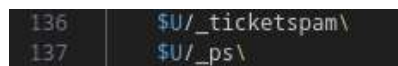
O presente trabalho busca alterar a implementação de um kernel Linux, mais especificamente, o kernel xv6, composto por um conjunto mínimo de instruções UNIX, adicionando novas chamadas de sistema para implementar um novo mecanismo de escalonamento baseado em tickets. Dessa forma, o Lottery Scheduler atua atribuindo tickets para os processos, e então o processo que possuir o valor sorteado é escolhido pelo escalonador.

2 Modificações necessárias

Foram necessários adicionar duas chamadas de sistema, 'settickets' para atribuir um certo número de tickets ao processo que realiza a chamada, a chamada 'getpinfo' que retorna informações sobre os processos em execução além de alterações no escalonador e em outras funções para adequação ao Lottery Scheduler. Dessa forma, as seguintes modificações foram feitas:

- Adicionar chamadas de sistema no Makefile

Para implementar as chamadas de sistema não foi necessário criar nenhum arquivo novo no diretório do kernel. Porém, como também foi feitos novos programas de usuário para execução das funções ('ticketspam' para executar a chamada 'settickets' e 'ps' para executar 'getpinfo'), o Makefile precisou ser alterado para permitir que esses arquivos fossem compilados, como mostrado abaixo:

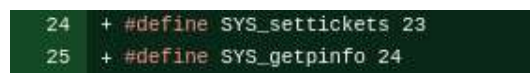


```
136 $U/_ticketspam\
137 $U/_ps\
```

Figure 1: Alterações no makefile

- Adicionando novos índices para o vetor de chamadas de sistema (xv6/kernel/syscall.h)

O arquivo "syscall.h" atua de maneira simples, definindo os números de cada chamada de sistema implementada. Olhando o arquivo, percebe-se que há uma definição para cada chamada já feita, logo, foi necessário adicionar apenas novas entradas com os novos números das *syscalls* 'settickets' e 'getpinfo' implementadas no trabalho. Tal ação é importante para a operação que será discutida a seguir.



```
24 + #define SYS_settickets 23
25 + #define SYS_getpinfo 24
```

Figure 2: Alterações syscall.h

- Alteração do arquivo com o vetor de chamadas de sistema (xv6/kernel/syscall.c)

O arquivo "syscall.c" é um dos principais responsáveis por coordenar a execução das chamadas de sistema no kernel do xv6. Primeiramente, nele são definidas as funções que representam cada *syscall*, que são organizadas em um array de ponteiros, em que cada posição (índice definido de acordo com o arquivo "syscall.h" comentado acima) mapeia o ponteiro da respectiva função da chamada de sistema, sendo necessário adicionar novas entradas nesse array

representação as novas funções implementadas. Nesse caso, as funções das novas chamadas de sistemas estão definidas no arquivo "sysproc.c" que será comentado mais abaixo.

```
133 [SYS_settickets] sys_settickets,  
134 [SYS_getpinfo] sys_getpinfo,  
135 ...
```

Figure 3: Alteração vetor de ponteiros.

- Definição da estrutura auxiliar pstat (xv6/kernel/pstat.h)

A estrutura 'pstat' é um dos pontos centrais da tarefa, sendo responsável por armazenar as informações do escalonamento dos processos através da estratégia de Lottery Scheduler. Assim, são salvos quais processos estão em uso atualmente (inuse), qual o seu respectivo valor de ticket (tickets), qual o pid de cada processo (pid) e quanto tempo de CPU cada processo já recebeu (ticks).

```
6 struct pstat {  
7     int inuse[NPROC];  
8     int tickets[NPROC];  
9     int pid[NPROC];  
10    int ticks[NPROC];  
11 };
```

Figure 4: Alterações pstat.h

- Implementação das chamadas de sistema 'settickets' e 'getpinfo' (xv6/kernel/sysproc.c)

Para implementar as chamadas de sistema, as funções responsáveis foram adicionadas no arquivo "sysproc.c". Assim, a função 'settickets' utiliza a estrutura 'pstat' comentada acima para coordenar os tickets dos processos. Inicialmente o número de tickets é recebido como argumento da função (argint) e então é salvo o PID do processo atual. Dessa forma, a lista de todos os processos em execução armazenada em 'pstat' é percorrida, e ao achar o processo atual e verificar que está em uso, o número de tickets associado a esse processo é alterado. Caso o número de tickets for inválido, a chamada retorna -1. Dentro da função, o número de total de tickets ('total_tickets') também é controlado, visto que apenas processos em uso são contabilizados para a contagem.

Já a função 'getpinfo' tem como objetivo retornar informações sobre os processos em execução. Para isso, basta retornar a estrutura pstat, que será usada no arquivo do nível de usuário 'ps' detalhada mais abaixo. Esse processo é feito então com a função 'copyout', que copia o espaço de memória desejado durante o retorno da chamada.

As alterações comentadas acima estão respectivamente evidenciadas nas imagens abaixo:

```
123 uint64 sys_settickets(void) {  
124     int num, pid;  
125  
126     argint(0, &num);  
127     pid = myproc()->pid;  
128  
129     if (num < 1) return -1;  
130  
131     // Lock so we can modify pstat and total tickets safely  
132     acquire(&tickets_pstat_lock);  
133  
134     // Set num tickets to calling process (pid)  
135     for(int p_idx = 0; p_idx < NPROC; p_idx++) {  
136         if (pstat.pid[p_idx] == pid && pstat.inuse[p_idx]) {  
137             // Update total tickets (don't count previous tickets)  
138             total_tickets += num - pstat.tickets[p_idx];  
139  
140             // Update process tickets  
141             pstat.tickets[p_idx] = num;  
142  
143             release(&tickets_pstat_lock);  
144             return 0;  
145         }  
146     }  
147  
148     release(&tickets_pstat_lock);  
149     return -1;  
150 }
```

Figure 5: settickets em sysproc.c

```
152 uint64 sys_getpinfo(void) {  
153     uint64 pstat_ptr;  
154     argaddr(0, &pstat_ptr);  
155  
156     // check nullptr  
157     if (pstat_ptr == 0)  
158         return -1;  
159  
160     // we need to use copyout to copy data from kernel to user space  
161     struct proc *p = myproc();  
162  
163     acquire(&tickets_pstat_lock);  
164     int rc = copyout(p->pagetable, pstat_ptr, (char*) &pstat, sizeof(struct pstat));  
165     release(&tickets_pstat_lock);  
166  
167     return rc;  
168 }
```

Figure 6: getpinfo em sysproc.c

- Declaração das chamadas de sistemas implementadas no espaço de usuário (xv6/user/user.h)

Com as chamadas de sistema totalmente implementadas no kernel, foi necessário declarar a assinatura no espaço do usuário para futuro uso em programas fora do espaço do kernel. Para isso, foram adicionadas as assinaturas dos novos métodos em "user.h", contendo exatamente as funcionalidades descritas previamente, possibilitando que programas de usuários importem essa definição para seu uso apropriado.

```
27 int settickets(int);
28 int getpinfo(struct pstat*);
```

Figure 7: Assinatura de settickets e getpinfo no espaço de usuário.

- Adição do *stub* para comunicação do espaço de usuário com o kernel (xv6/user/usys.pl)

O arquivo "usys.S" é responsável por delegar as assinaturas de chamadas de sistema declaradas no espaço do usuário para as chamadas de sistema em si, presentes no kernel. Esse arquivo é gerado pelo script "usys.pl", que define o código assembly para a passagem de parâmetros correta e a chamada da syscall em si, transferindo o controle para o kernel. Com isso, foram adicionadas as entradas das novas chamadas de sistema implementadas 'settickets' e 'getpinfo' no mesmo padrão do que as chamadas definidas originalmente.

```
40 entry("settickets");
41 entry("getpinfo");
```

Figure 8: Stub adicionado em usys.pl

- Alterações em *allocproc*, *fork*, *freeproc*, *exit*, *wakeup*, *sleep* e *scheduler* (xv6/kernel/proc.c)

O arquivo 'proc.c' é o principal arquivo alterado nesse trabalho. Nele são modificados pontos importantes do kernel, ligados a criação, alteração e escalonamento dos processos para adequar o funcionamento do sistema ao novo método de escalonamento implementado. Os tickets são salvos em uma variável global chamada 'total_tickets', sendo usada em todo o arquivo. Começando pela função 'allocproc', ela é responsável por definir o PID e cuidar da organização da memória de novos processos. Assim, foi adicionado em seu código a inicialização da estrutura 'pstat' para o novo processo, sendo setado o 'inuse', recebendo também 1 ticket durante sua criação e o 'ticks' é 0 pois o processo acabou de ser criado. Com isso, sua posição na estrutura é o valor do PID do processo criado, como visto na figura abaixo. Vale destacar que para sabermos o índice do processo criado na estrutura 'pstat', foi utilizado aritmética de ponteiros para calcular o deslocamento do processo atual na estrutura de todos os processos.

```
163 // Pointer math to get index of process in pstat
164 int p_idx = p - proc;
165
166 // Process was born, init pstat for that process
167 acquire(&tickets_pstat_lock);
168 pstat.inuse[p_idx] = 1;
169 pstat.tickets[p_idx] = 1;
170 pstat.pid[p_idx] = p->pid;
171 pstat.ticks[p_idx] = 0;
172
173 total_tickets++;
174 release(&tickets_pstat_lock);
```

Figure 9: Alteração allocproc

Já na função 'fork', como na especificação do trabalho foi definido que processos criados com 'fork' deveriam receber o mesmo número de tickets do processo pai, bastou-se definir o número de tickets do 'pstat' do filho como o mesmo valor do pai. Vale destacar que como 'allocproc' já incrementa em 1 o número total de tickets e que dentro do 'fork' a função 'allocproc' é chamada, então não é necessário esse novo incremento, por isso ocorre a subtração de -1. Como

o 'fork' é utilizado juntamente do 'exec' e o 'fork' já copia os tickets do pai para o filho, então não foram necessárias alterações no 'exec'. Da mesma forma, como o 'exec' executa outro comando a partir do processo pai, então os tickets continuam os mesmos para o novo processo, não sendo necessário mudanças.

```
360 // Pointer math to get index of process in pstat
361 int p_idx = p - proc;
362 int np_idx = np - proc;
363
364 // Set child tickets equal to parent tickets
365 acquire(&tickets_pstat_lock);
366 pstat.tickets[np_idx] = pstat.tickets[p_idx];
367
368 // Increment total tickets
369 total_tickets += pstat.tickets[p_idx] - 1; // Need to sub one cause allocproc incremented one already
370 release(&tickets_pstat_lock);
371 release(&np->lock);
```

Figure 10: Alterações fork

Outra modificação foi na função 'freeproc' que libera algum processo existente. Assim, a adição feita foi apenas setar o 'inuse' na estrutura 'pstat' como 0, já que o processo estará liberado. Já na função 'exit', foi necessário fazer alterações para liberar a contagem de tickets quando um processo é finalizado. Dessa forma, o número de tickets do processo atual é subtraído do total de tickets. A mesma ideia do 'exit' é aplicada na função 'sleep', pois um processo que está dormindo não precisa ter sua contagem de tickets considerada. Analogamente, na função 'wakeup', ao acordar a contagem de tickets do processo volta a ser levada em consideração. Vale destacar que a função 'kill' que também está relacionada ao fim de processos não teve nenhuma modificação, visto que dentro do arquivo (/kernel/trap.c), os processos que são mortos chamam o 'exit', e como o 'exit' já lida com os tickets, não foram necessárias alterações. Todas essas mudanças estão evidenciadas nas imagens abaixo.

```
200 // Pointer math to get index of process in pstat
201 int p_idx = p - proc;
202
203 // Process has ended, so set its inuse flag to 0
204 acquire(&tickets_pstat_lock);
205 pstat.inuse[p_idx] = 0;
206 release(&tickets_pstat_lock);
207 }
```

Figure 11: Alterações freeproc

```
429 // Pointer math to get index of process in pstat
430 int p_idx = p - proc;
431
432 acquire(&tickets_pstat_lock);
433 total_tickets -= pstat.tickets[p_idx];
434 release(&tickets_pstat_lock);
```

Figure 12: Alterações exit

```
679 // Pointer math to get index of process in pstat
680 int p_idx = p - proc;
681
682 // Runnable process has tickets
683 acquire(&tickets_pstat_lock);
684 total_tickets += pstat.tickets[p_idx];
685 release(&tickets_pstat_lock);
```

Figure 13: Alterações wakeup

```
648 // Pointer math to get index of process in pstat
649 int p_idx = p - proc;
650
651 // Sleeping process has no tickets
652 acquire(&tickets_pstat_lock);
653 total_tickets -= pstat.tickets[p_idx];
654 release(&tickets_pstat_lock);
```

Figure 14: Alterações sleep

A função 'scheduler' precisou ter sua lógica alterada para adicionar o novo método de escalonamento. Primeiro, um "número aleatório" é sorteado entre 1 e o total de tickets (a função de sorteio está definida em xv6/kernel/rand.h e apenas sorteia um número pseudo aleatório no intervalo desejado). Com isso, ao selecionar o novo processo a ser escalonado, é verificado qual processo possui o ticket vencedor, o que é feito percorrendo 'pstat', enquanto é salvo o número de tickets vistos até o momento em 'counted_tickets'. Assim, ao passar pelo vencedor, a troca de contexto acontece normalmente, e após ser finalizado, o seu número de 'ticks' é atualizado. Todas as alterações podem ser vistas nas imagens abaixo.

```

// Xorshift Pseudo Random Number Generator.
// Returns a number between 0 and RAND_MAX_UNSIGNED, both inclusive.
// Adapted from: https://www.jstatsoft.org/article/view/v008i14
uint32 rand() {
    static uint32 x = 1234567890, y = 3624360690, z = 5212886290, w = 886751230;

    uint32 t;
    t = (x * (x << 11));
    x = y;
    y = z;
    z = w;
    return (w = (w * (w >> 19)) ^ (t * (t >> 8)));
}

// Ranged Pseudo Random Number Generator.
// Returns a number between min and max, both inclusive.
// Number returned will be ≤ than RAND_MAX_SIGNED.
// Adapted from: https://stackoverflow.com/questions/2509679/how-to-generate-a-ra
int rand_range(int min, int max) {
    // max ≤ RAND_MAX_SIGNED < RAND_MAX_UNSIGNED, so this is okay.
    uint32 num_bins = (uint32) max + 1 - min;
    uint32 num_rand = (uint32) RAND_MAX_SIGNED + 1;
    uint32 bin_size = num_rand / num_bins;
    uint32 defect = num_rand % num_bins;

    int x;
    do {
        x = (int) rand();
    }

    // This is carefully written not to overflow
    while (num_rand - defect ≤ (uint32) x);

    // Truncated division is intentional
    return x / bin_size + min;
}

```

Figure 15: Geração de números aleatórios

```

int winner = rand_range(1, total_tickets);
release(&tickets_pstat_lock);

int counted_tickets = 0;

// Loop through all processes to find winner
for(p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);

    int p_idx = p - proc;

    acquire(&tickets_pstat_lock);
    int p_inuse = pstat.inuse[p_idx];
    release(&tickets_pstat_lock);

    if (p->state == RUNNABLE && p_inuse) {
        // Increment counted and check if winner has been chosen
        acquire(&tickets_pstat_lock);
        counted_tickets += pstat.tickets[p_idx];
        release(&tickets_pstat_lock);

        if (counted_tickets ≥ winner) {
            // Switch to chosen process. It is the process's job
            // to release its lock and then reacquire it
            // before jumping back to us.
            p->state = RUNNING;
            c->proc = p;

            // Save start ticks
            int tick_start = ticks;

            // Switch
            switch(&c->context, &p->context);

            // Process is done running for now.
            // It should have changed its p->state before coming back.
            c->proc = 0;

            // Get delta ticks
            acquire(&tickets_pstat_lock);
            pstat.ticks[p_idx] += ticks - tick_start;
            release(&tickets_pstat_lock);

            // Run external loop again, picking a new winner
            release(&p->lock);
            break;
        }
    }

    release(&p->lock);
}

```

Figure 16: Escalonamento com base nos tickets

- Arquivos de teste das chamadas de sistema (xv6/user/ticketspam.c e xv6/user/ps.c)

Para fins de teste e uso geral das chamadas de sistema, foram feitos dois programas em nível de usuário que interages com as *syscalls*, utilizando a assinatura dos métodos definidas previamente em "user.h". O método 'ticketspam' funciona chamando a *syscall* 'settickets', passando como parâmetro o número de tickets informado ao programa. Assim, caso algum parâmetro informado for inválido, o programa irá tratar o erro. Caso o número de tickets for válido, o código então simplesmente continua sua execução, enquanto o processo ainda estiver vivo.

Já o método 'ps' atua chamando a *syscall* 'getpinfo', que percorre toda a lista de processos na estrutura 'pstat', imprimindo na tela as informações coletadas: PID, TICKETS e TICKS. Novamente, caso algum erro ocorrer, o código retorna para o usuário o problema na execução. Ambas as implementações estão evidenciadas nas imagens abaixo:

```

1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user/user.h"
4 #include "kernel/pstat.h"
5
6 int main(int argc, char *argv[]) {
7     if (argc != 2) {
8         fprintf(2, "Usage: ticketspam [number of assigned tickets]\n");
9         exit(1);
10    }
11
12    int res = settickets(atoi(argv[1]));
13
14    if (res == -1) {
15        fprintf(2, "ticketspam: could not set tickets to process.
16        Number of tickets cannot be smaller than 1\n");
17        exit(1);
18    }
19    else {
20        for (;;)
21            ;
22    }
23    exit(0);
}

```

Figure 17: Função ticketspam.c

```

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/pstat.h"

// User space program that prints processes status
int main(int argc, char *argv[]) {
    struct pstat pstat;

    if (getpinfo(&pstat) == -1) {
        fprintf(2, "ps: could not get process info\n");
        exit(1);
    }

    printf("PID    TICKETS    TICKS\n");

    for (int i = 0; i < NPROC; i++) {
        if (pstat.inuse[i])
            printf("%d    %d    %d\n", pstat.pid[i],
                pstat.tickets[i], pstat.ticks[i]);
    }

    exit(0);
}

```

Figure 18: Função ps

3 Testes

Para a testagem, foi-se utilizada a função implementada no arquivo "/user/testtickets.c", que basicamente cria três processos com alguma proporção definida (no teste abaixo, a proporção foi 1:2:3) fazendo a leitura das suas informações (com a função 'getpinfo') por algum período determinado. Dessa forma, é possível ver a evolução do escalonamento de cada processo ao longo do tempo. Vale destacar que os testes foram feitos executando o trabalho com o comando **make qemu CPUs=1**, assim apenas uma CPU é utilizada, o que facilita a visualização dos resultados e as diferenças no escalonamento dos processos, respeitando assim o número de tickets aplicados. Para utilização da função, basta iniciar o qemu e rodar o comando **testtickets <numero de leituras> <tickets do testtickets> <proporção desejada>** que cria um csv com as leituras, com o gráfico plotado abaixo para favorecer a visualização. Outro ponto importante sobre o código desenvolvido é a presença de diferentes travas (mutex) para controlar o acesso a 'pstat' e ao contador global de tickets. Isso é importante para evitar problemas de execução envolvendo mais CPUs, que podem alterar ou ler trechos de memória em diferentes momentos, podendo haver impactos com a estrutura que coordena o escalonamento.

Com isso, a imagem abaixo representa a evolução dos 'ticks' (tempo de CPU já escalonado para cada processo) no eixo Y e no eixo X mostra-se as diferentes leituras feitas dos processos. Percebe-se então como o processo em verde que possui 3 tickets, apresentou maior número de 'ticks' do que os demais, sendo o processo em azul o menos escalonado, já que possui apenas 1 ticket. Percebe-se então que o escalonamento está sendo feito como deveria, já que o número de 'ticks' está diretamente ligado a quantidade de tickets que cada processo possui.

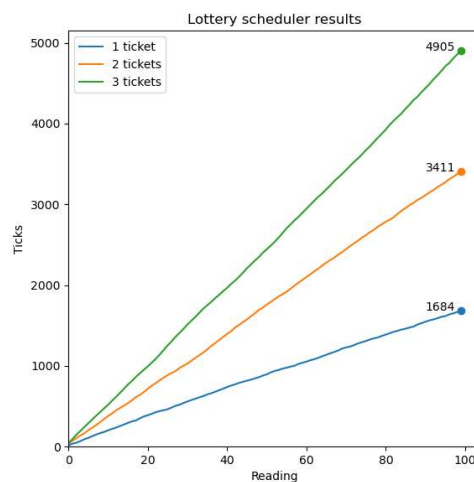


Figure 19: Gráfico de evolução do número de 'ticks' para processos com diferentes tickets

Outro teste feito para comprovar a boa execução do Lottery Scheduler envolve atribuir a um processo um certo

número de tickets, e a outro um numero maior para checar sua evolução. Dessa forma, como mostrado na figura abaixo, de início o processo com 1 ticket de PID=4 foi escalonado mais vezes do que o processo com PID=6 com 10 tickets (visto que o primeiro processo foi criado antes). Porém, com o tempo, o segundo processo vai sendo escalonado mais vezes, representado pelo maior número de 'ticks' que ultrapassa o primeiro processo. Com isso, mais uma vez é evidenciado a funcionalidade desejada na descrição do trabalho.

```
init: starting sh
$ ticketspam 1 &
$ ticketspam 10 &
$ ps
PID      TICKETS    TICKS
1         1         0
2         1         0
7         1         0
4         1        191
6        10       185
$ ps
PID      TICKETS    TICKS
1         1         0
2         1         0
8         1         0
4         1       235
6        10       610
$ ps
PID      TICKETS    TICKS
1         1         0
2         1         0
9         1         0
4         1       359
6        10      1736
$ ps
PID      TICKETS    TICKS
1         1         0
2         1         0
10        1         0
4         1       468
6        10      2842
$ █
```

Figure 20: Teste contagem ticks

4 Referências

ChatGPT não foi utilizado.

Repositório Xv6 public

Repositório Xv6 riscv

GeekForGeeks - Lottery Scheduling

Repositório com orientações para o trabalho

Geração de números aleatórios em um intervalo

Geração de números aleatórios